

PASK: Cold Start Mitigation for Inference with Proactive and Selective Kernel Loading on GPUs

Xuanteng Huang*, Jiangsu Du[†], Nong Xiao[‡], Xianwei Zhang[§]

School of Computer Science and Engineering

Sun Yat-sen University

Guangzhou, China

*huangxt57@mail2.sysu.edu.cn, {[†]dujiangsu, [‡]xiaon6, [§]zhangxw79}@mail.sysu.edu.cn

Abstract—Today, DNN inference is widely adopted, with numerous inference services being spawned from scratch across instances in scenarios such as spot serving, serverless scaling and edge computing, where frequent start-stops are required. In this work, we first delve into the inference workflow and uncover the origins of cold start when invoking a DNN model. Specifically, DNN execution is blocked by the kernel loading process to prepare the code object executing on GPU at the DL primitive library (e.g., cuDNN and MIOpen). To tackle this, we propose PASK, a kernel loading and reusing middleware to mitigate the widespread cold start issue. Unlike the reactive kernel scheduling policy used by existing frameworks, PASK adopts a proactive strategy to interleave code loading, kernel issuing and GPU computation to achieve higher hardware utilization. To further reduce the loading overhead, PASK recycles existing loaded kernels to accomplish the DNN operator, rather than introducing new kernels for every layer. Meanwhile, PASK categorically organizes the cached kernels to efficiently find the applicable kernel for reuse and thus minimize incurred runtime overhead. We implement and evaluate PASK atop of open source DNN inference engine and primitive library on off-the-shelf GPUs. Experiments demonstrate PASK is capable of alleviating the cold start overhead of popular DNN models with $5.62\times$ speedup on average.

Index Terms—Cold Start, GPU, DNN, Inference

I. INTRODUCTION

The last decade has witnessed the unprecedented prosperity of deep learning and its widespread application in many aspects like computer vision [1]–[9], recommendation [10], and natural language processing [11]. Serving model inference has become the dominating workload in modern data centers, which accounts for more than 90% infrastructure costs in AWS [12] and 200 trillion daily invocations in Meta [13].

This trend also sparks the research efforts devoted into the ML inference infrastructures like request batching [14]–[17], model selection [18], [19], memory optimizations [20], tensor pre-fetching [21] and resource sharing [22], [23]. As underlying support, the accelerators like GPUs are trending to be much more powerful and the DNN operators are continuously optimized by experts for efficient computation [24]–[27]. As a result, DNN inference tends to be more hindered by the non-computation aspects, with a significant portion being spent on the models’ cold starting process [28]. Fig. 1(a) compares the cold and hot execution times (ratios between the first and successive iterations) of representative DL models on both data center and consumer-grade GPUs from different vendors, where significant cold start slowdowns are commonly observed on respect platforms ($23.7\times$, $19.5\times$ and $31.3\times$ for MI100, A100 and

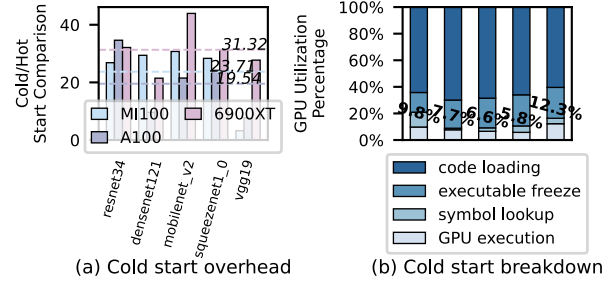


Fig. 1. DNN model cold start (a) overhead and (b) breakdown of typical models. The dashed lines in (a) represents averaged slowdowns and percentage numbers in (b) mark the GPU execution portion.

6900XT, respectively). Fig. 1(b) further demonstrates the breakdowns of the cold start overhead, which is classified into four parts by their execution ordering phases. Before launching a kernel onto the GPU, the host runtime (i.e., CUDA or HIP) is responsible for checking whether the kernel code object (e.g., compiled SASS instructions binary stream) is present in the managed host memory. If not, the runtime will load it from either the shared library or compiled ELF executables, then set memory permissions and look up the target symbol positions. It can be captured from Fig. 1(b) that the majority (65.8%) of cold start overhead is attributed to loading absent kernel code objects (code loading) while the computation (GPU execution) constitutes only a small proportion (8.4%). Therefore, the code loading should be considered alongside data pre-fetching [21], keep alive [29] and pre-warming [30] techniques to mitigate DNN cold start overhead.

The cold start overhead originated from code loading is inevitable in scenarios like preemptive serving [31]–[36], serverless scaling [17], [20], [29], [37] or edge computing [30], [38], [39]. For example, the inference service has to be migrated across preemptive instances, or deployed in new instances to cope with request spikes. In edge or mobile devices, the inference service is forced to be suspended or swapped out due to limited hardware resources (e.g., memory capacity), and restarts later. Under these circumstances, code loading is unavoidable when starting a DNN model as there are no available loaded kernels in the system.

There are multiple proposed approaches to mitigate the non-computation overhead of DNN models [25]–[27], [40]. TASO [25], Rammer [26], Cocktail [27] and MonoNN [40] apply some operator-level transformations on the models’ computation graph for higher execution efficiency. Nevertheless, these methods focus on the static optimizations to produce a series of fast kernels running on GPUs, failing to consider the crucial runtime overhead of loading these kernels. There also exist designs sharing container [29], [41], runtime [30], [42] or tensor [20] to warm up the cold models, but

[§] Xianwei is the corresponding author.

This research was supported by the National Key R&D Program of China under Grant NO. 2022ZD0115304, the National Natural Science Foundation of China #62472462, #62461146204, #62402534, #62332021, the Guangdong Basic and Applied Basic Research Foundation: 2023A1515110117, and was sponsored by CCF-Tencent Rhino-Bird Open Research Fund (CCF-Tencent RAGR20240102).

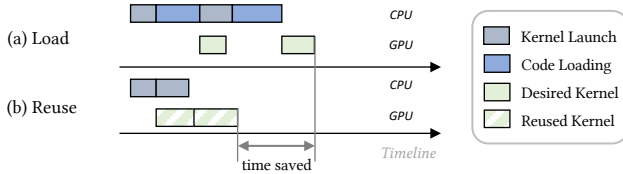


Fig. 2. Instead of waiting for the fresh loading (a), PASK reuses existing loaded kernels (b) to implement the DNN layers.

the sharing granularity is generally too coarse as DNN models are monolithic and cannot be efficiently transformed via dependency analysis for traditional services [43]. As such, extra steps are needed to transform the shared resources for accommodating with the cold start model. NNV12 [38] optimizes the DNN cold start process via selecting different kernels to implement the NN operator to reduce the tensor layout interchange overhead before execution. Nonetheless, the above existing methods ignore the kernel code object loading procedure, which actually constitutes the most significant portion of the cold start overhead for DNN models (Fig. 1).

In this work, we propose PASK, a kernel loading and reusing approach to mitigate the cold start latency of DNN inference. We first analyze the DNN inference serving procedure and attribute the cold start overhead to the mismatch between static model materialization and runtime kernel loading: inference framework determines the finalized kernel sequences from abstract model computation graph without considering the incurred loading overhead at runtime. To cope with this, PASK seeks the opportunities to *reuse* existing potentially less performant but already loaded kernels to materialize NN operator, thereby skipping the more substantial kernel loading procedure, as depicted in Fig. 2.

However, it's non-trivial to efficiently select the substitute from all cached kernels as checking whether the kernel is applicable for current problem can be pretty time-consuming. To this end, we design a categorical cache which stores the cached kernels based on their patterns to minimize the applicable checking overhead. We integrate PASK into the open source inference stack with AMD GPUs¹, and conduct evaluations on various DNN models which show remarkable performance improvements.

To summarize, this work makes the following contributions:

- We investigate the origins of DNN model cold start overhead, and uncover it is primarily due to the mismatch between static model materialization and runtime kernel loading.
- With the insights, we propose PASK, a kernel loading and reusing approach leveraging the already loaded kernels to finalize the NN operator, thus circumventing the substantial kernel loading process. Equipped with interleaved execution and categorical cache, PASK achieves higher GPU utilization meanwhile minimizing the incurred runtime overhead.
- We implement PASK atop of open source inference software stack, and evaluate using representative DNN models to demonstrate its ability to mitigate the cold start overhead.

II. BACKGROUND AND MOTIVATION

A. DNN Model Serving with GPU

GPU has become the *de-facto* accelerator to speed up DNN execution. A DNN model is composed of multiple layers with each defining a specific operation like convolution or pooling. To harness the hardware resources, DNN operators are offloaded onto GPUs for

¹This can be implemented on NVIDIA platforms as well if more flexible interfaces are accessible.

faster execution through various frameworks like TensorRT [44] for NVIDIA and MIGraphX [45] for AMD GPUs. These frameworks *materialize* the DNN layers with the computation graph optimizations [46]–[48] and tuned kernels [24], [49] for efficient execution.

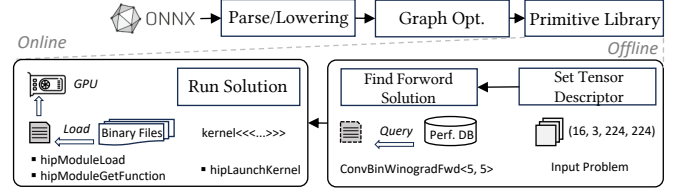


Fig. 3. DNN inference serving workflow. The serving framework firstly lowers the model and applies multiple optimizations on the requested model, then invokes operators from the primitive library to materialize the model.

Offline preparation. Fig. 3 depicts a typical workflow of DNN serving with GPU, where the requested model is submitted in the ONNX format containing multiple canonical operators [50] for the ease of model description and exchange. After receiving the model, the serving framework first converts the ONNX structure into lower descriptions with regard to the backend hardware, including GPU context creation, host-device synchronization setup, and tensor memory layout generation. Then the serving framework applies a series of hardware-independent graph-level optimization passes like *dead code elimination* and *common subexpression elimination* [45], [51] to reduce unnecessary computations. Finally, the serving framework exploits the highly-optimized kernels from vendor-provided primitive libraries (e.g., MIOpen and cuDNN) to accomplish the operators defined in the computation graph.

For each DNN layer (e.g., convolution), the serving framework sets the tensor descriptors needed by the primitive library with input *problem* (image and filter sizes, number of filters, data types etc.). Then the library finds the optimal *solution*² for the given problem from all applicable ones by querying an integrated database [52] which records the anticipated performance of each solution on current problem. After this, the lower operations and the determined optimal solutions for each layer are saved in framework-specific formats (e.g., `.trt` files for TensorRT and `.mgx` files for MIGraphX) for online inference request serving. The framework usually maintains a model registry [53] to store the lowered model and directly loads them when the request comes to avoid redundant lowering.

Online inference serving. When the inference request arrives, the framework invokes the corresponding optimized model and runs the determined solutions sequentially by launching the underlying kernels onto the GPU. Nevertheless, there are often cases where the code object of the desired kernel is absent due to the lazy loading behavior [54] which delays the loading until launch. Therefore, the primitive library has to *load* the code objects for the missing kernels from compiled binary files (e.g., dynamic linked ELF), resulting in unbearable cold start latency for model inference. Only after the code objects are loaded into memory will the kernels be issued to the GPU for computation. Such a reactive launch and lazy loading behavior leads to substantial latency at model cold start. Note that the loading delay comes from the mismatch between the offline operator lowering and the runtime solution loading: the primitive library determines solutions from the GPU performance perspective, meanwhile introducing extra loading overhead when launching missing kernels. In

²One solution may contain multiple kernels to transform input/output tensor layout/precision, conduct primary computation and epilogue reduction.

principle, the gap can be effectively bridged if one can proactively interleave the code loading and kernel launch executions, and reuse already loaded kernels to accomplish the computation rather than loading missing kernels from scratch each time.

B. Generalized Solution for Wider Applicability

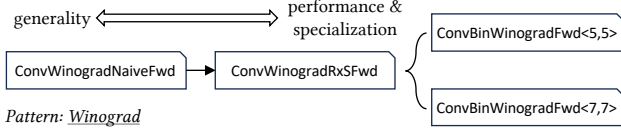


Fig. 4. Generality-performance trade-offs for *Winograd* solutions. Specialized solutions are more performant but applicable for specific problems, while generic solutions can be applied to more problems but are less efficient.

Solution specialization. Fig. 4 demonstrates some exemplified convolution solutions embedded in MIOpen. The solution usually follows certain *patterns* to conduct the computation. For example, solutions using *Winograd* [55] accomplish the convolution via matrix multiplication, and apply some transformations on both the input image and filters to lessen the number of multiply-and-add instructions. Other commonly used patterns include *GEMM*, *DirectConv* and *ImplicitGEMM*. Solutions with the same pattern typically follow similar workflows to accomplish the computation. Besides, there are further optimization spaces to exploit if certain problem fields are deterministic before the model starts (i.e., compile time). In Fig. 4, instead of taking the *ConvWinogradNaiveFwd* solution which accepts inputs with any dimension, one can alter to invoke *ConvBinWinogradRxFwd* if the input is deemed to be a 2D image. By avoiding unnecessary loops and boundary checks, the latter hence brings in higher GPU performance, but at the expense of generality. Furthermore, the library opts for leveraging more specialized solution *ConvBinWinogradFwd<5,5>* if the filter size is also a specific value (5×5), which helps organize the shared memory layout and better overlap the memory access and computation within the kernel. The highly specialized solutions produce better GPU performance tailored for the specific problem, but they are not applicable to other problems as there are more constraints. Currently, the primitive libraries (cuDNN, MIOpen) tend to harness the specialized solutions from the performance perspective, and fallback to the more general solutions only if no specialized solution is located. Inevitably, this leads to more solution loads at the model cold start, as the library may select an uniquely specialized solution for each individual layer with varying problem description.

Solution applicability. To ensure successful executions and correct results of DNN inference, the primitive library should only return *applicable* solutions for a given problem. The applicability checking process is time-consuming as it includes complicated logic such as workspace size checking, input format match, environment variable and hardware capability validation. In MIOpen, each solution implements the `IsApplicable` interface for the library to check whether it can be used to accomplish the computation for a given problem without any constraint violations (e.g., data type mismatch, out-of-bound memory access). Apparently, the incurred runtime overhead can be significantly alleviated if the applicable checks can be reduced to quickly identify a valid substitutive solution.

III. DESIGN

In this section, we present the components and workflow of PASK. Fig. 5 illustrates the overview of our design, where PASK receives the lowered model as its input. During model parsing, PASK chooses

to proactively load the solutions once they are parsed and starts the execution immediately once they are loaded (section III-A). For an absent solution, PASK checks the applicability of loaded solutions to seek the opportunities of reusing existing solution to accomplish the computation, thereby skipping to load the missing one (section III-B). Upon reusing, PASK integrates a categorical solution cache (section III-C) to organize existing loaded solutions and provides an interface to efficiently select an applicable solution for the given problem. Finally, we elaborate the implementation of PASK atop of open source inference engine and DL primitive library (section III-D).

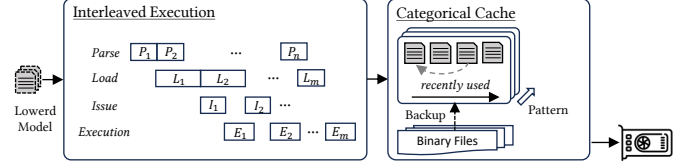


Fig. 5. Design overview. PASK receives the lowered model (a sequence of n solutions) as input, and then proactively parses, loads, issues the DNN layer to improve the GPU utilization via interleaved execution. Upon absent solution, PASK seeks the opportunities to reuse the loaded solutions and organizes them based on their patterns for efficient applicability checking.

A. Proactively Interleaved Execution

We introduce the interleaved execution to cope with the inefficient execution hindered by the reactive launch and lazy loading pattern mentioned in section II-A. During parsing layers in DNN computation graph, PASK spawns two extra host threads to handle code loading and kernel issuing, respectively (Fig. 5). For a layer i , once it gets parsed (P_i , de-serialize the framework-specific model file to obtain the determined solution), the loading thread immediately starts to load the code objects of the desired solution (L_i). Once finish, the loading thread notifies the issuing thread to issue the kernels of the loaded solution (I_i) and execute them on GPU (E_i). This proactive design loads the necessary code objects in advance (once the parse finishes), therefore launches of the corresponding kernels incur no loading overhead. Besides, PASK starts the kernels at the earliest feasible time right after the loading. For comparison, current reactive serving frameworks start to launch the solutions only after all layers are parsed, and load the absent code objects on demand during launch. PASK mitigates the inefficiency with proactive interleaved execution, thereby achieving higher GPU utilization than traditional reactive inference engines.

As the DNN layer parsing is much faster than the code loading and execution, there will be a layer m where all the n layers are parsed and all preceding layers, including m , have finished their executions on GPUs (Fig. 5). We denote layer m as the *milestone* where PASK unconditionally loads the missing solutions into cache set $\mathcal{S}_m$ before m and selectively reuses the cached solutions after m . Note the minimal duration of a model execution is no less than the parsing process, hence there are free time slots for PASK to load before m solutions, which act as cached alternatives for the selective reuse in subsequent layers.

B. Selective Solution Reuse

After the milestone layer m , PASK quickens the execution of the remaining layers by avoiding unnecessary solution loads with selective adoption of cached solution. *Algorithm 1* lists the solution reusing procedure where the `EXECUTIONPLAN` function describes the layer-wise reuse decision procedure. When the target solution s_i^* ($i > m$) in layer i is parsed, PASK checks whether the corresponding

code object has been already loaded. If so, s_i^* will be directly used to complete the computation defined in layer i without incurring any loading overhead. Otherwise, PASK firstly attempts to acquire the applicable alternative solution by querying the cached solution set $\mathcal{S}_m$ (section III-C). Once a valid substitutive solution $\hat{s}_i$ exists, PASK will reuse it to accomplish the computation in layer i rather than loading the original absent solution s_i^* from scratch.

If, unfortunately, no valid solution can be found in the cached set $\mathcal{S}_m$, PASK has no choice but to load the desired solution s_i^* .

Algorithm 1 DNN execution with solution reuse

Input:
 n : number of layers in the DNN model
 m : the milestone layer (section III-A)
 $\mathcal{S}_m$: solution cache set at layer m
 p_i : problem to be solved in layer i
 s_i^* : statically optimal solution in layer i

```

1: function EXECUTIONPLAN( $\mathcal{S}, L, i$ )
2:   for layer  $i$  from  $m+1$  to  $n$  do
3:     if  $s_i^* \in \mathcal{S}_m$  then                                ▷  $s_i^*$  already loaded
4:       compute layer  $i$  with  $s_i^*$ 
5:     else
6:        $\hat{s}_i \leftarrow \text{GETSUBSOLUTION}(\mathcal{S}, s_i^*, p_i)$ 
7:       if  $\hat{s}_i \neq \text{null}$  then                                ▷ reusable solution found
8:         run layer  $i$  with substitutive solution  $\hat{s}_i$ 
9:       else                                                ▷ no applicable solution
10:        wait for  $s_i^*$  loading
11:        add  $s_i^*$  to  $\mathcal{S}_m$ 
12:   function GETSUBSOLUTION( $\mathcal{S}, s^*, p$ )
13:     list  $\leftarrow \mathcal{S}[s^*.pattern]$ 
14:     for  $s \in$  list's most recently used solution do
15:       if  $s.\text{IsApplicable}(p)$  then
16:         lru.update( $s$ )
17:         return  $s$ 
18:   return null

```

C. Categorical Solution Cache

In *Algorithm 1*, PASK queries the solution cache for a loaded solution to be reused for implementing the operator. As discussed in section II-B, it is expensive to lookup the applicability with current problem for every loaded solution in the cache. PASK thus adopts a categorical solution cache design to effectively find an applicable solution from all existing loaded ones.

In PASK, the loaded solutions are categorically organized in separated lists based on their *patterns* (section II-B). When a solution is loaded/invoked, it will be inserted/moved to the head of list. The GETSUBSOLUTION function in *Algorithm 1* details how PASK handles the query for a substitutive solution given the desired solution s^* and problem p . It seeks for $\hat{s}$ from the list with the same pattern as s^* in the the most recently accessed (load or lookup) order. If one applicable solution is found in the categorical list, it is returned and moved to the list head (Fig. 5). Otherwise, the cache returns *null*, indicating there are no applicable solutions for current problem.

PASK adopts this categorical design to minimize the number of lookups (i.e., solution applicability checking for a problem) when searching for $\hat{s}$. When a specialized solution for problem p is missing, it is more likely to find a more general one with the same pattern to solve p (Fig. 4). If there are no applicable solution in one categorical list, PASK regards the query failed and returns *null* directly to skip more lookups for other categories. We search the solutions in the recent access order similar to the LRU cache, as the neighboring layers in DNN models usually possess similar problem descriptions. Therefore, recently used solutions are more likely to be applicable to current problem.

D. Implementation

We implement PASK on the basis of the open source inference engine MIGraphX [45] and DL primitive library MIOpen [56] for

TABLE I
EVALUATED DNN MODELS.

Model	# Primitive Layers	Type	Abbr.
AlexNet [1]	5	Img. Rec.	alex
VGG16 [2]	16	Img. Rec.	vgg
ResNet34 [3]	14	Img. Rec.	res
RegNet_Y_800MF [58]	28	Img. Rec.	reg
EfficientNet_B7 [4]	58	Img. Rec.	eff
Faster_R-CNN [5]	16	Obj. Det.	rcnn
SSD300 [59]	27	Obj. Det.	ssd
FCN [6]	18	Sem. Seg.	fcn
UNet [60]	37	Sem. Seg.	unet
ViT_B_16 [7]	1	ViT	vit
Swin_B [8]	1	ViT	swin
Swin_V2_B [9]	1	ViT	swin2

AMD GPUs. PASK acts as a transparent middleware for end users.

We enhance MIGraphX to support the interleaved execution (section III-A). MIGraphX abstracts the computation graph by a list of instructions, which define the DNN operators and their corresponding input/output tensors. For interleaved execution, PASK spawns 3 host threads responsible for parsing, loading and issuing kernels, respectively (Fig. 5). We leverage single-producer-single-consumer (SPSC) channels to coordinate the interactions among host threads in MIGraphX. Instructions are fed to the loading thread when finishing the parse and leaves for the interleaved execution in the FIFO order. For each callee kernel of the solution, PASK invokes `hipModuleLoad` and `hipModuleGetFunction` (Fig. 3) to proactively load the target ELF sections from the binary files and determine location of the desired symbol by the kernel name.

We further augment MIOpen to support the solution reuse and loading skip in PASK. When the inference framework invokes `miopenRunSolution`, PASK follows *Algorithm 1* to reuse the potentially applicable loaded solution for the computation in current layer. PASK implements the reuse logic for the common convolution, pooling and activation primitives in MIOpen.

IV. EXPERIMENT SETUP

Testbed. We evaluate PASK in a sever equipped with AMD EPYC 7773X CPU and AMD Instinct MI100 GPU, which possesses 32 GB VRAM and 120 Compute Units. The DNN inference server is implemented atop of MIGraphX and MIOpen as the graph optimization engine and DL primitive library backed by ROCm 6.0.2.

Workloads. We collect representative DNN workloads (Table I) from multiple fields (e.g., image recognition, object detection, semantic segmentation and vision transformer) implemented the PyTorch model zoo as served models in inference requests, where the vision transformer models are also used to emulate the transformer-based language models. Models are exported as ONNX format from PyTorch under the default configuration and then fed into MIGraphX for compute graph optimizations, and finally lowered to the corresponding GPU operations. The number of MIGraphX layers for each model is also listed in Table I. We follow the common image settings used in the ImageNet [57] dataset.

Evaluated schemes. To faithfully validate the effectiveness of PASK, we evaluate the above models under the following schemes:

- *Baseline*: the default inference workflow used by MIGraphX inference framework which loads every absent solutions.
- *NNV12*: the cold start mitigation design in NNV12 [38] to avoid input/output tensor layout transform.
- *Ideal*: hot execution of the DNN model, with all the desired solutions already resident.
- *PaSK*: full design of PASK, including interleaved execution and categorical solution cache.
- *PaSK-I*: PASK with only interleaved execution module being enabled, no solution reuse mechanism is enabled.

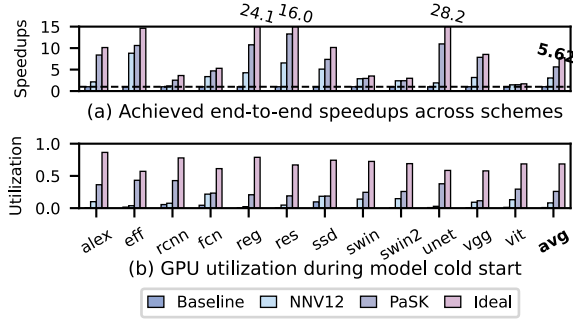


Fig. 6. (a) End-to-end cold start speedups for evaluated models under different schemes; (b) GPU utilization (fraction of time when GPU is active for computing) during model cold start.

- *PaSK-R*: PASK with solution reusing only, which exhaustively checks the applicability of every cached solution.

V. RESULTS AND ANALYSIS

A. Cold Start Mitigation

We first investigate the achieved cold start mitigation effect under different schemes. Fig. 6(a) compares the end-to-end cold start speedups under *Baseline*, *NNV12*, *PaSK* and *Ideal* schemes, where *PaSK* achieves $5.62\times$ speedups over *Baseline*. Compared to *Baseline*, *NNV12* possesses only moderate speedups ($3.04\times$). *NNV12* mitigates the cold start by avoiding tensor layout transforms (e.g., NCHW $\rightarrow$ NHWC) between layers, limiting the optimize opportunities on specific layers. Furthermore, we can observe that *PaSK* performs better on models with more primitive layers (eff, unet, reg and ssd), as there are more opportunities for PASK to reuse the existing substitutive solution to replace the absent one to accomplish the computation. On the other hand, transformer models (vit, swin and swin2) perceive less end-to-end cold start latency improvement compared to other convolution-based models. As transformer models primarily comprise of GEMM operators from the BLAS library, there is only one primitive layer provided by MIOpen (Table I). Hence, there are less reuse chances as PASK could be invoked only when the primitive routine is called.

TABLE II
COLD START SPEEDUP WITH VARYING INFERENCE BATCH SIZES.

Batch Size	1	4	16	64	128
<i>NNV12</i>	$3.04\times$	$2.82\times$	$2.49\times$	$1.91\times$	$1.74\times$
<i>PaSK</i>	$5.62\times$	$5.24\times$	$4.55\times$	$3.91\times$	$3.10\times$
<i>Ideal</i>	$7.75\times$	$7.23\times$	$6.94\times$	$6.78\times$	$6.41\times$

We further demonstrate the cold start improvements with varying batch size for each inference request in Table II. With enlarged batch size, GPU is better utilized and the proportion of duration consumed on code loading is thereby lower. Besides, hot execution is still far more efficient compared to cold start, while *PaSK* still outperforms *NNV12* with a large margin (210%).

B. Utilization and Breakdown

To understand the effectiveness of PASK, we adopt the hardware utilization (the proportion of time when GPU is active relative to the entire inference duration) as the metric to explain how PASK boosts the cold start performance of DNN models on GPU. Fig. 6(b) illustrates the achieved utilization under the various schemes, where we can capture that the GPU active time accounts for only a few portion (8.2%) of model execution for *NNV12*. As the majority of

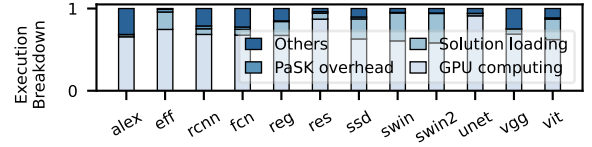


Fig. 7. Model cold start breakdown for *PaSK*.

time is spent on kernel code loading, the cold start latency is several times higher than the one in hot start. *PaSK* avoids this substantial overhead by reusing the applicable solutions from rather than always loading the missing one, and effectively elevates the utilization to 25.9% on average. For *Ideal*, since all solutions are already cached, there is no loading overhead when launching the desired kernels onto the GPU thus the accomplished utilization is up-to 68.5% on average.

Figure 7 further depicts the breakdowns of times consumed on different stages during model execution. We compare the ratios of times spent on GPU computing, solution loading, PASK overhead and others (e.g., host-device synchronization, model parse). The portion of time consumed on solution loading (11.2% on average) is acceptable compared to the cold start execution without dedicated management (Fig. 1). For transformer models (vit, swin, swin2), the solution loading proportion is relatively larger than other models, as the major operations are GEMMs. Moreover, the extra time incurred by PASK takes only 1.3% of the entire execution on average, indicating that PASK can efficiently retrieve the applicable alternative solution with negligible overhead.

C. Ablation Studies and Cache Statistics

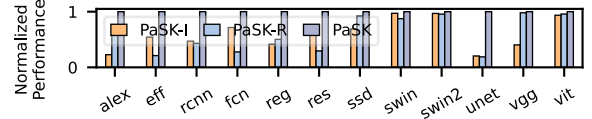


Fig. 8. Normalized performance of variants compared to *PaSK*.

In this section, we compare *PaSK* with its variants *PaSK-I* and *PaSK-R* to validate the effectiveness of solution reusing and categorical cache. For *PaSK-I*, the cold start latency reduction comes from the interleaved execution. Regarding *PaSK-R*, it adopts the naive cache which exhaustively searches all cached solution to find the optimal one, suffering from spending too much time in the applicability checking which is time-costly (section II-B). Fig. 8 illustrates the achieved proportional performance normalized to *PaSK*. For models perform worse in *PaSK-I* (alex, vgg, unet), we find the GPU execution durations before the milestone layer are short, thus there are less opportunities for *PaSK-I* to overlap the loading and execution process in these models. And we can only observe nuances of performance for transformer models, as they contain only one DL primitive operator (Table I) thereby possess few reuse opportunities.

To further investigate the impact of categorical cache, we collect the cache statistics during cold start. The cache hit rate describes the averaged possibility of successfully finding a applicable solution following the GETSUBSOLUTION routine in *Algorithm 1*. Fig. 9(a) shows the hit rate of categorical cache in PASK across models (69.7% on average) where transformer models are omitted as they contain only one primitive operator. We can capture that models comprising of more operators possess higher hit rates, as there are more chances for PASK to load the desired solution into the categorical cache following *Algorithm 1*. For subsequent queries, it is more likely to find a applicable solution in a cache with more entries.

Fig. 9(b) presents the averaged number of lookups per query (i.e., number of solutions whose applicability are checked), where the

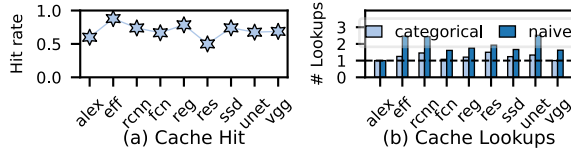


Fig. 9. Cache hit rate and averaged number of solution lookups for each hit.

categorical organization only uses 1.22 lookups per query while the naive searching requires 1.89 on average. As introduced in section II-B, checking the solution’s applicability for a given problem is time-costly. Therefore, the naive solution searching strategy may result in severe performance downgrade due to enormous applicable checking overhead. It can be drawn from Fig. 9(b) that models with more lookups in the naive cache (eff, unet, res, rcnn) suffer from more pronounced performance downgrade in *PaSK-R* in Fig. 8. And the categorical organization in PASK can effectively reduce the number of required lookups per query, thus further boost the achieved performance.

VI. DISCUSSIONS

Library supporting. As discussed earlier, the vision transformer models (vit, swin, swin2) possess less performance boosts from PASK because the major computation is matrix multiplication in the attention module [61]. Currently PASK is implemented atop of MIOpen, which contains the primitive operations used in DNN models (e.g., convolution, pooling), while the GEMM kernels are provided by the BLAS library (e.g., hipBLAS). Therefore, PASK can only identify and reuse a small portion of layers in the transformer models and offers limited cold start latency reduction for them. Note that it is trivial to extend PASK to support GEMM operators and accommodate with transformer models if more engineering efforts can be devoted to apply the similar modifications to hipBLAS. As hipBLAS and other vendor libraries (e.g. hipSPARSE [62] for sparse operators in GPU) following the same find-execution pattern (Fig. 3).

More factors for kernel specialization. Furthermore, PASK could be combined with other mechanisms by considering more factors. For instance, recent researches [63], [64] leverage the mix-precision components integrated in accelerators [65] to speedup the inference since DNN models usually have better tolerance on significance loss and numeric error. As the DNN operators are also specialized for tensor data types (fp16, int8), one may choose to still use high-precision data types if the corresponding kernels are already loaded while the low-precision ones are not. This offers the opportunities to accelerate the inference with high-precision computation by avoiding the overhead to load the absent kernels tailored for low-precision data types, even though the latter provide better performance on GPU. PASK can be generalized and applied to scenarios where the GPU kernels are specialized for some factors like input/output layout, data type, tensor tile size.

Loading desired solutions. Although PASK selectively skips the originally desired solutions to avoid the loading overhead, one can still load the skipped solutions during the intervals between two consecutive inference requests. Based on cloud workload traces [66], there are several seconds on average between the interval of two requests scheduled into the same inference instance. The interval duration is sufficient for PASK to proactively load the previously skipped solutions into the managed cache. Synergistically, they can act the candidates for reuse, or the desired solution which are already loaded for subsequent requests.

VII. RELATED WORKS

A. Cold Start Mitigation of DNN Models

Mitigating the cold start of DNN model has become the research appetite in recent years. Optimus [29] tackles the ML serverless container cold start problem by proposing the inter-function model transform approach which reuses operators from models of idle containers to compose the target model. Tetris [20] improves the serverless DNN inference latency and system container density via reusing duplicated tensors across instances. Pagurus [41] designs the algorithm to calculate the software dependency similarities between containers and then harness the idle container to build the intermediate image ready for helping to start the container rather than building from scratch. NNV12 [38] selectively loads the transformed weights in advance to avoid the tensor layout interchange overhead, and distributes the load and computation to different hardware units on edge/mobile devices.

These existing works share the container [29], [37], [41], service runtime [30], or model weights [20] among models. However, DNN models monolithic and cannot be transformed separately. Hence, some extra steps are needed to transform the shared resources to accommodate with the target model (e.g., modify the structure of warm models [29]). There are also general cold start mitigation approaches designed for cold microservice or serverless environments [37], [41], [67]–[70], but they fail to investigate the inference workflow and propose dedicated optimizations for DNN model invocations.

B. DNN Inference Optimizations

As the new computing paradigms thrive in recent years, cold start mitigation approaches have been proposed for preemptive spot [31], [33], [35], [41] computing and serverless computing [37]. For DNN model inference services under these two circumstances, the cold start process is inevitable as the serving instance has to be migrated across instances, or spawned in a new environment to satisfy the resource demands. PASK can be further integrated into the spot serving and serverless frameworks to mitigate the cold start issues with dedicated kernel loading management.

To harness the GPU capability for accelerating DNN computation, performance experts attempt to optimize operator graphs and model workflows to alleviate the memory bandwidth bottlenecks. TASO [25], Rammer [26], Cocktail [27] and MonoNN [40] are graph transformation methods applying the intra- and inter-operator optimizations for the better hardware utilization. However, these approaches solely consider the kernel efficiency on GPU, and ignore the overhead of loading the multifarious generated kernels. PASK can be used to enhance these methods by assisting with some runtime information (e.g., the loaded kernels) to generate a more efficient execution plan for cold models.

VIII. CONCLUSION

In this work, we investigate the DNN model inference procedure and attribute cold start latency to the reactive scheduling strategy and kernel loading overhead. In order to mitigate the cold start latency, we propose PASK, a transparent middleware to schedule model kernels in a proactive strategy, and seek the opportunities to reuse existing loaded solutions organized categorically. We implement PASK on basis of the open source inference framework and primitive library and evaluate PASK with several popular DNN models. Detailed experiments demonstrate that PASK can effectively alleviate the cold start overhead for DNN model inference with $5.62\times$ speedups compared to the default workflow used by the inference frameworks.

REFERENCES

- [1] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *NeurIPS*, 2012.
- [2] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- [3] Kaiming He et al. Deep residual learning for image recognition. In *CVPR*, 2016.
- [4] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.
- [5] Shaoqing Ren et al. Faster R-CNN: towards real-time object detection with region proposal networks. *PAMI*, 2017.
- [6] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.
- [7] Alexey Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [8] Ze Liu et al. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, pages 9992–10002, 2021.
- [9] Ze Liu et al. Swin transformer V2: scaling up capacity and resolution. In *CVPR*, 2022.
- [10] Maxim Naumov et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv*, 2019.
- [11] Jacob Devlin et al. BERT: pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, 2019.
- [12] Deliver high performance ML inference with AWS Inferentia. https://d1.awsstatic.com/events/reinvent/2019/REPEAT_1_Deliver_high_performance_ML_inference_with_AWS_Inferentia_CMP324-R1.pdf.
- [13] Michael J. Anderson et al. First-generation inference accelerator deployment at facebook. *arXiv*, 2021.
- [14] Ahsan Ali et al. Batch: machine learning inference serving on serverless platforms with adaptive batching. In *SC*, 2020.
- [15] Pin Gao et al. Low latency RNN inference with cellular batching. In *EuroSys*, 2018.
- [16] Gyeong-In Yu et al. Orca: A distributed serving system for transformer-based generative models. In *OSDI*, 2022.
- [17] Chengliang Zhang et al. Mark: Exploiting cloud services for cost-effective, slo-aware machine learning inference serving. In *ATC*, 2019.
- [18] Sohaib Ahmad et al. Proteus: A high-throughput inference-serving system with accuracy scaling. In *ASPLOS*, 2024.
- [19] Daniel Mendoza, Francisco Romero, and Caroline Trippel. Model selection for latency-critical inference serving. In *EuroSys*, 2024.
- [20] Jie Li et al. Tetris: Memory-efficient serverless inference through tensor sharing. In *ATC*, 2022.
- [21] Yangyang Feng et al. Mobius: Fine tuning large-scale models on commodity GPU servers. In *ASPLOS*, 2023.
- [22] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *EuroSys*, 2024.
- [23] Kelvin K. W. Ng, Henri Maxime Demoulin, and Vincent Liu. Paella: Low-latency model serving with software-defined GPU scheduling. In *SOSP*, 2023.
- [24] Tianqi Chen et al. TVM: an automated end-to-end optimizing compiler for deep learning. In *OSDI*, 2018.
- [25] Zhihao Jia et al. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *SOSP*, 2019.
- [26] Lingxiao Ma et al. Rammer: Enabling holistic deep learning compiler optimizations with rtasks. In *OSDI*, 2020.
- [27] Chen Zhang et al. Cocktail: Analyzing and optimizing dynamic control flow in deep learning. In *OSDI*, 2023.
- [28] Zhihao Bai et al. Pipeswitch: Fast pipelined context switching for deep learning applications. In *OSDI*, 2020.
- [29] Zicong Hong et al. Optimus: Warming serverless ML inference via inter-function model transformation. In *EuroSys*, 2024.
- [30] Misun Park, Ketan Bhardwaj, and Ada Gavrilovska. Pocket: ML serving from the edge. In *EuroSys*. ACM, 2023.
- [31] Jiangfei Duan et al. Parcae: Proactive, liveput-optimized DNN training on preemptible instances. In *NSDI*, 2024.
- [32] Xupeng Miao et al. Spotserv: Serving generative large language models on preemptible instances. In *ASPLOS*, 2024.
- [33] Sanjith Athlur et al. Varuna: scalable, low-cost training of massive deep learning models. In *EuroSys*, 2022.
- [34] Luo Mai et al. Kungfu: Making training in distributed machine learning adaptive. In *OSDI*, 2020.
- [35] John Thorpe et al. Bamboo: Making preemptible instances resilient for affordable training of large dnn. In *NSDI*, 2023.
- [36] Insu Jang et al. Oobleck: Resilient distributed training of large models using pipeline templates. In *SOSP*, 2023.
- [37] Yanan Yang et al. Inffless: a native serverless system for low-latency, high-throughput inference. In *ASPLOS*, 2022.
- [38] Rongjie Yi et al. Boosting DNN cold inference on edge devices. In *MobiSys*, 2023.
- [39] Kaihua Fu et al. Qos-aware irregular collaborative inference for improving throughput of DNN services. In *SC*, 2022.
- [40] Donglin Zhuang et al. Mononn: Enabling a new monolithic optimization space for neural network inference tasks on modern gpu-centric architectures. In *OSDI*, 2024.
- [41] Zijun Li et al. Help rather than recycle: Alleviating cold startup in serverless computing through inter-function container sharing. In *ATC*, 2022.
- [42] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. Daydream: Executing dynamic scientific workflows on serverless platforms with hot starts. In *SC*, 2022.
- [43] Hanfei Yu et al. Rainbowcake: Mitigating cold-starts in serverless with layer-wise container caching and sharing. In *ASPLOS*, 2024.
- [44] NVIDIA TensorRT. <https://developer.nvidia.com/tensorrt>.
- [45] AMD MIGraphX Documentation. <https://rocm.docs.amd.com/projects/AMDMIGraphX/en/latest/>.
- [46] Size Zheng et al. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In *HPCA*, 2023.
- [47] Ganesh Bikshandi and Jay Shah. A case study in CUDA kernel fusion: Implementing flashattention-2 on NVIDIA hopper architecture using the CUTLASS library. *arXiv*, 2023.
- [48] Ao Li et al. Automatic horizontal fusion for GPU kernels. In *CGO*, 2022.
- [49] Introduction to torch.compile — PyTorch Tutorials. https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html.
- [50] ONNX Operators - ONNX documentation. <https://onnx.ai/onnx/operators/>.
- [51] Optimization — NVIDIA Triton Inference Server documentation. https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton_inference_server_1150/user-guide/docs/optimization.html.
- [52] Using the performance database — MIOpen 3.2.0 Documentation. <https://rocm.docs.amd.com/projects/MIOpen/en/latest/conceptual/perfdb.html>.
- [53] Model Repository — NVIDIA Triton Inference Server. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_repository.html.
- [54] Nathan Otterness and James H. Anderson. Exploring AMD GPU scheduling details by experimenting with “worst practices”. *Real Time Syst.*, 2022.
- [55] Andrew Lavin and Scott Gray. Fast algorithms for convolutional neural networks. In *CVPR*, 2016.
- [56] MIOpen documentation. <https://rocm.docs.amd.com/projects/MIOpen/en/latest/>.
- [57] Jia Deng et al. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [58] Ilija Radosavovic et al. Designing network design spaces. In *CVPR*, pages 10425–10433. Computer Vision Foundation / IEEE, 2020.
- [59] Wei Liu et al. SSD: single shot multibox detector. In *ECCV*, 2016.
- [60] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.
- [61] Ashish Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.
- [62] hipSPARSE documentation. <https://rocm.docs.amd.com/projects/hipSPARSE/en/latest/>.
- [63] Ji Lin et al. AWQ: activation-aware weight quantization for on-device LLM compression and acceleration. In *MLSys*, 2024.
- [64] Yilong Zhao et al. Atom: Low-bit quantization for efficient and accurate LLM serving. In *MLSys*, 2024.
- [65] NVIDIA Tensor Cores: Versatility for HPC & AI. <https://www.nvidia.com/en-us/data-center/tensor-cores/>.
- [66] Qizhen Weng. Beware of fragmentation: Scheduling gpu-sharing workloads with fragmentation gradient descent. In *ATC*, 2023.
- [67] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. Icebreaker: warming serverless functions better with heterogeneity. In *ASPLOS*, 2022.
- [68] Ashraf Mahgoub et al. ORION and the three rights: Sizing, bundling, and prewarming for serverless dags. In *OSDI*, 2022.
- [69] Zijun Li, Quan Chen, and Minyi Guo. Pagurus: Eliminating cold startup in serverless computing with inter-action container sharing. *arXiv*, 2021.
- [70] Dong Du et al. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *ASPLOS*, 2020.

To Tackle Cost-Skew Tradeoff: An Adaptive Learning Approach for Hub Node Selection

Guowei Sun^{*2}, Lin Chen^{*1}, Qiming Huang³, Hu Ding^{†1}

¹School of Computer Science and Technology, ²School of Data Science, ³School of the Gifted Young
University of Science and Technology of China, Hefei 230026, China
{gwsun1998, ischenlin, chnhqm}@mail.ustc.edu.cn, huding@ustc.edu.cn

Abstract—In chip design, *skew* is a pivotal factor that significantly influences the overall performance for routing. A major challenge is how to achieve an appropriate trade-off between the total wire-length cost and skew. Selecting hub nodes is an effective method to improve this cost-skew trade-off. In this paper, we propose a novel reinforcement learning-based method for hub node selection, where our key idea is leveraging an effective adaptive learning strategy. Moreover, our approach is particularly suitable for solving large-scale routing instances. The empirical results suggest that our method can achieve promising performance on both small-scale and large-scale clock nets, implying its potential practical significance in EDA.

Index Terms—Skew, wirelength, MSPD.

I. INTRODUCTION

In the past decades, the problem of designing efficient algorithms for various VLSI routing applications has attracted substantial attention [1]–[6]. The routing quality are closely related to several crucial metrics in chip design, which are the so-called “PPA”, i.e., *performance*, *power*, and *area* [7]. The design procedure of interconnect trees relies on several widely studied optimization objectives, among which the total tree **cost** is the most prevalent, typically measured by wire length. Additionally, the **skew**, defined as the difference in signal travel time between the longest and shortest root-leaf paths, is also a crucial factor, as illustrated in Fig. 1. For different applications, these objectives may have different priorities.

In this paper, we focus on designing an effective routing algorithm for tackling the tradeoff between the wire-length and skew. For chip design, the “clock frequency” is an important performance indicator, as it determines the number of instructions that the processor can execute per second [8]. The clock frequency is the reciprocal of the clock cycle, which can be affected by several different factors in practice. In particular, the clock skew often seriously influences the clock cycle; as noted in [9], it can account for even more than 10% of the cycle time. Consequently, how to effectively address the clock skew issue is an imperative problem that we are confronted with for designing high-quality chips.

Actually, the skew issue is a long historical topic in the research of EDA. Several classical methods were proposed before, such as the methods of *Zero-Skew Tree* (ZST) and *Bounded-Skew Tree* (BST) [8]. The goal of ZST is to build a

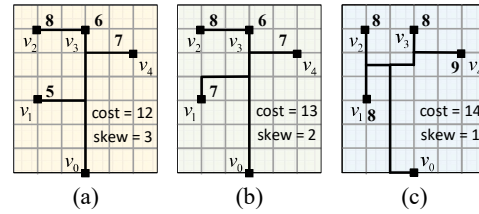


Fig. 1: This illustration shows that different routing priorities can lead to various topologies; v_0 is designated as the root node, and the number marked alongside each vertex is the distance to v_0 in the tree. Figure (a) depicts a design focusing on minimizing wire length [8], while Figure (b) illustrates a trade-off between cost and skew. Figure (c) primarily emphasizes skew, aiming to minimize the time difference for signals traveling from the root node to other nodes.

routing tree where the signal from the root node can reach all the terminals at the same time, so that the skew is zero. Boese and Kahng [10] presented the Deferred-Merge Embedding (DME) algorithm for building a ZST that can accomplish precise zero skew. However, a drawback of ZST is that it often results in long wire-length and it is difficult to be realized due to the limitations of the manufacturing process [8]. This motivates researchers to consider an alternative solution, Bounded-Skew Tree [11]–[13], which is a clock tree with a maximum skew bounded by a given threshold $UB > 0$. It is worth noting that when there is no skew restriction (i.e., $UB = \infty$), the BST problem is simply equivalent to the classical Rectilinear Steiner Minimum Tree (RSMT) problem [14]. Several heuristic methods were developed to address the bounded skew Steiner tree construction problem in the past decades [12], [13]. Cong et al. [11] provided an improved DME algorithm called “BST-DME”, which is specifically designed for the BST problem. Additionally, several elegant approximation algorithms were also proposed for the skew problem [15], [16]. More recently, Han et al. [17] studied the tradeoff between cost and skew and formulated the bounded skew problem as a flow-based integer linear programming.

Recently, building upon the classic Prim-Dijkstra (PD) algorithm [18], Kahng et al. [19] introduced the **Multi-Source Prim-Dijkstra** (MSPD) algorithm for the cost-skew tradeoff problem, which will be detailed in Section II-B. The main insight of the MSPD approach is to select several centrally located “hub” nodes (termed “sources” in their paper) to serve as root connections and enforce a constraint that prevents other nodes from connecting directly to the root. Subsequently, the

^{*}First two authors contributed equally. [†]Corresponding author. This work was supported in part by the Fundamental Research Funds for the Central Universities through Grant WK2150110025.

modified PD algorithm (i.e., “MSPD”) is employed to generate the tree topology. From the examples illustrated in Fig. 2, we can see the advantage of utilizing hub nodes.

However, there is still a gap for efficiently identifying the hub nodes. **The first challenge** comes from the high computational cost, since an optimal algorithm would require an exhaustive search for the best combination of hub nodes. Although the method proposed in [19] could reduce the complexity to a certain extent by only considering those centrally located node candidates, the resulting smaller solution space could affect the final performance of routing. Moreover, **the second challenge** is that there is currently no effective method capable of adaptively deciding how many hub nodes should be chosen. The existing algorithms require specifying a maximum quantity of hub nodes, and then enumerating different combinations that have the sizes within such an upper bound. Due to the importance of hub nodes in the skew problem, the open-source project *OpenROAD* (“Foundations and Realization of Open, Accessible Design”) in collaboration with TILOS AI Institute recently launched an innovation-driven competition in June 2023, whose objective is to devise efficient hub node selection algorithms to achieve favorable cost-skew tradeoff.

To tackle those challenges in addressing the cost-skew tradeoff problem, we propose an adaptive learning-based framework. In particular, our framework takes account of the global space for determining the number of chosen hub nodes and their locations. **The main contributions** of our paper are outlined as follows:

- 1) We propose a novel “Selector $\rightleftharpoons$ Conductor” idea that can be seamlessly integrated with the existing reinforcement learning method. The “Selector” network is for selecting hub nodes, and the “Conductor” network is used for monitoring the current environment and actions to determine when to halt operations. This cooperation mechanism between them yields an adaptive learning process, and also enhances our model’s ability to manage larger-scale instances.
- 2) To evaluate the performance of our proposed method, we conducted a set of experiments on both small-scale and large-scale problems using diverse datasets sourced from synthetic data and the OpenROAD competitions. In the experiments, our learning-based method achieved the best Pareto frontier in terms of cost and skew across various test cases and demonstrated satisfying efficiency.

II. PRELIMINARIES

A. Problem Formulation

An instance of a clock routing problem comprises n nodes, denoted as $V = \{v_0, v_1, \dots, v_{n-1}\}$, with v_0 acting as the **root node** and other nodes serving as the **sinks**. The goal is to construct a routing tree such that the signal from the root, v_0 , can be transmitted to the other nodes. Suppose a routing tree $T = (V, E)$ is constructed to connect all the nodes in V . In this routing tree T , let $e_{i,j}$ denote the edge linking v_i

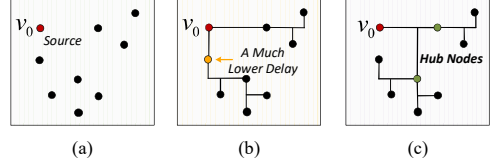


Fig. 2: This figure illustrates the role of hub nodes. Figure (a) shows the locations of all nodes. Figure (b) depicts a routing tree without adding hub nodes, leading to significant skew; the orange sink experiences a much lower delay than the other black sinks. Figure (c) presents a routing tree generated by the MSPD algorithm, demonstrating a reduction in skew through the aid of hub nodes.

with v_j ; its edge cost is defined as the Manhattan distance $|x_j - x_i| + |y_j - y_i|$ (suppose $v_i = (x_i, y_i)$ and $v_j = (x_j, y_j)$). Consequently, the **cost** of the clock tree, denoted as $\text{cost}(T)$, is the sum of all edge costs, i.e.,

$$\text{cost}(T) = \sum_{e_{ij} \in E} |x_j - x_i| + |y_j - y_i|. \quad (1)$$

The clock **skew** refers to the difference between the highest and lowest arrival times of the clock signal across the $n - 1$ sinks. Formally, we define:

$$\text{skew}(T) = \max_{v_i, v_j \in V} |t(v_0, v_i) - t(v_0, v_j)|, \quad (2)$$

where $t(v_0, v_i)$ denotes the signal delay between the nodes v_0 and v_i , estimated by using either the *linear delay* or the *Elmore delay* model [8]. The linear delay model equates delay directly with the path length between v_0 and v_i , ignoring other details in the circuit, whereas the Elmore model is more complex that also takes the resistance and capacitance into account. Due to its simplicity for calculation, the linear model remains popular in commercial tools [8].

As discussed in Section I, a central problem for practical clock routing is designing an effective algorithm for achieving a pleasant **cost-skew tradeoff**. Formally, given a node set V and a parameter $\alpha \in [0, 1]$, our objective is to construct a routing tree T to minimize:

$$\alpha * \text{cost}(T) + (1 - \alpha) * \text{skew}(T). \quad (3)$$

B. Multi-Source Prim-Dijkstra (MSPD)

Multi-Source Prim-Dijkstra (MSPD) [19] is a variant of the PD algorithm [18]. Given a clock routing instance as shown in Fig. 2(a), a major difference of MSPD is that the algorithm selects a set of hub nodes and only these designated nodes can link directly to the root. The purpose of this constraint is to avoid large skew (see the comparison between Fig.2(b) and Fig.2(c)). It is noteworthy that the authors of [19] proposed two algorithms for hub node selection. The first one is an optimal algorithm; however, it incurs a significantly high computational cost due to its exhaustive search for the best combination of hub nodes. To avoid such a large combination space, the authors also introduced an alternative approach that only enumerates the combinations of the potential hub nodes located centrally to the root; the “centrality” is measured by several pre-specified parameters (we refer the reader to

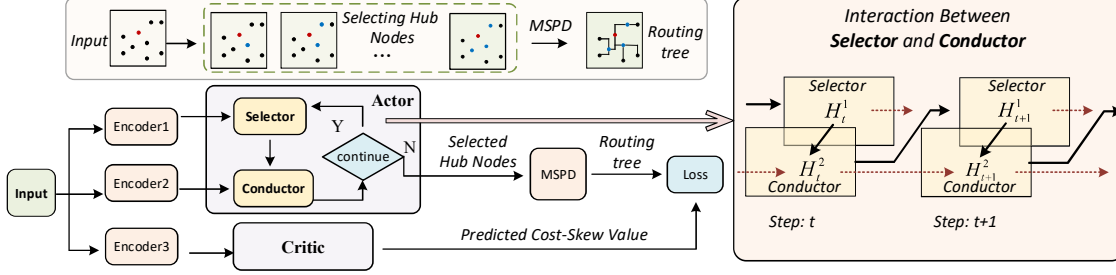


Fig. 3: Our proposed RL-based framework, whose core neural components are the Selector and Conductor networks. The Selector network determines the hub node in each step, while the Conductor network monitors the environment and existing actions, assessing the need for further actions. The right figure illustrates the detailed interaction between these networks; the black arrow indicates the execution order, while the dotted brown arrow represents information passed from previous steps. A running example is also shown in this figure (top left): given an input node set, the red node is the root, and the hub nodes (labeled in blue) can be determined by the Selector and Conductor networks. Then, the instance is processed by the MSPD algorithm and eventually, the routing tree with cost-skew tradeoff is obtained.

their article [19] for more details). Although this method reduces the size of the candidate set, it results in a narrower solution space that could downgrade the obtained solution quality. Moreover, the prior methods usually require specifying a maximum quantity of hub nodes, which may not be easy to determine and thus also restricts the capability to optimize the objective of (3). These practical issues motivate us to develop an efficient and adaptive method for hub nodes selection in Section III.

C. Reinforcement Learning

Our proposed method utilizes Reinforcement learning (RL), which is an important machine learning methodology that has been extensively studied for many different real-world applications [20]. RL uses an **agent** to perform the learning process, where the agent can make decisions by taking actions in a specified environment. This RL process consists of interacting with the environment, evaluating the current state, and receiving rewards based on the outcomes of its actions. In particular, the **Actor-Critic** RL framework [20] has been successfully applied to solve several routing tasks in EDA [4], [5]. The “Actor” network typically involves a sequence of decision-making steps, such as connecting a node to the existing tree; ultimately this network will generate a routing tree. Meanwhile, the “Critic” network is employed to facilitate the training of the Actor network.

For completeness, we introduce the three important terminologies of RL that will be used throughout Section III: 1) **State**: At any given time step, the state includes the locations of all nodes in V , the root node v_0 , and the currently selected set of hub nodes; 2) **Action**: At each step t , the agent selects a new hub node that has not been previously chosen; subsequently, the agent must also determine whether to continue selecting additional hub nodes; 3) **Reward**: In our model, the reward is computed at the conclusion of the RL process and is combined with the critic’s prediction to assess the quality of the selected hub nodes.

Our method also follows the above overall framework; however, our proposed method needs to select specific hub nodes

during the tree construction, necessitating some significant new ideas for this purpose.

III. OUR PROPOSED FRAMEWORK

In this section, we introduce our proposed approach. First, we analyze the technical challenges and provide an overview of our framework below.

The challenges and our high-level idea. We are aware of several recently proposed RL-based methods for routing problems, such as the method of [4] for minimizing the wire-length and the method of [5] for minimizing the timing-driven objective. Nevertheless, we are still confronted with some new significant challenges in addressing the issue of cost-skew tradeoff. Both of those methods are designed for generating the tree using a Markov chain, with the aid of some clearly defined termination criteria. But in our problem, our aim is to select several key hub nodes. The immediate issues for realizing such a model include how to handle the uncertainty of the hub nodes and how to overcome the absence of a predefined termination criterion (note that even the number of needed hub nodes is also unknown in advance). Thus, our task is to design some flexible and easy-to-implement method that can adaptively select the hub nodes and efficiently construct the routing tree with a qualified cost-skew tradeoff.

Our high-level idea relies on a novel “**Selector** $\rightleftharpoons$ **Conductor**” reinforcement learning framework. To identify the most appropriate candidates for hub nodes, we design a “Selector” network that is dedicated to hub node selection. More importantly, we also integrate a “Conductor” module into the classical Actor module, which can help the model monitor the environment and existing actions, so as to determine the necessity of further actions. The architecture of our whole model is built upon this adaptive Selector $\rightleftharpoons$ Conductor mechanism, as shown in Fig. 3.

We present the technical details in the following parts. In Section III-A, we introduce the Encoder module that is used for representing the input data. The “Selector” and “Conductor” modules are two key parts to realize the adaptive learning in our framework, which are respectively introduced

in Section III-B and Section III-C. Finally, we describe the Critic network and the training method in Section III-D.

Superscripts of the notations. For ease of presentation, in the following sections, we use the superscripts from $\{0, 1, 2, 3\}$ to differentiate the parameters and variables in different modules. For example, W^0 denotes the parameters in the Encoder modules, W^1 is for the Selector network, W^2 is for the Conductor network, and W^3 is for the Critic network.

A. Encoder

The goal of the Encoder is to embed an input instance V (which contains n vertices in $2D$) into a higher dimensional space $\mathbb{R}^d$ with some appropriately set value d ; this new representation will serve the following modules as shown in Fig. 3. Our Encoder model is based on the seminal **attention mechanism** in deep learning [21], which is inspired by human visual attention, enables the model to weigh the importance of different inputs and allows the model to focus on different parts of the input sequence. Note that we have three encoders for the Selector, Conductor, and Critic networks, respectively (as shown in Fig. 3), where they have the same structure but with different trained parameters.

Given the routing instance V containing n nodes, we build the input to our encoder that is the form of a matrix $X_{\text{ini}} \in \mathbb{R}^{n \times 3}$. The 3-dimensional information includes the 2D coordinate information (x, y) and a label indicating whether it is a root node (labeled as 1 if it is a root node, and 0 otherwise). Then we embed these nodes into a higher dimensional space for enriching their encoded information: the input vector $X_{\text{ini}} \in \mathbb{R}^{n \times 3}$ is processed through a feed-forward neural network layer, obtaining the output $\hat{X}_{\text{ini}} \in \mathbb{R}^{n \times d_0}$ (where the integer d_0 is a pre-specified hyperparameter in our model). Formally, our feed-forward neural network can be calculated as:

$$\hat{X}_{\text{ini}} = \max \{ \vec{0}, X_{\text{ini}} \times W_1^0 + \vec{b}_1^0 \} \times W_2^0 + \vec{b}_2^0, \quad (4)$$

where $\vec{0}$ is the zero vector, and $W_1^0 \in \mathbb{R}^{3 \times d_0}$, $W_2^0 \in \mathbb{R}^{d_0 \times d_0}$, $\vec{b}_1^0 \in \mathbb{R}^{d_0}$, $\vec{b}_2^0 \in \mathbb{R}^{d_0}$ are the parameters to be trained in this feed-forward layer. Considering the importance of the root node, we need to ensure that the information of the root node v_0 should be accessible to other nodes. For this purpose, we concatenate the embedding of the root node with that of each other node: the obtained new embedding is $X \in \mathbb{R}^d$ ($d = 2d_0$), and each row

$$X[i] = [\hat{X}_{\text{ini}}[i], \hat{X}_{\text{ini}}[0]] \quad (5)$$

is the augmented embedding of the node v_i , where $\hat{X}_{\text{ini}}[i]$ is the i -th row of $\hat{X}_{\text{ini}}$, i.e., the original embedding of v_i , and $\hat{X}_{\text{ini}}[0]$ is the original embedding of the root.

Then, the embedding X is fed into the “multi-head” attention layer [21]. The function of this layer is to aggregate and capture information from all the nodes. Through this multi-head attention layer, the information from various nodes is aggregated to create a new representation for each node, and thus an enriched embedding can be obtained. As mentioned at

the beginning of this subsection, we have this Encoder network before each of the downstream Selector, Conductor, and Critic networks. We denote the corresponding outputs from these three Encoders as E^1 , E^2 , and $E^3 \in \mathbb{R}^{n \times d}$, respectively.

B. Selector Network

The function of the Selector network is to identify the nodes that have high potential to become the hubs. The methodology for identifying hub nodes leverages the **Pointer Network** [22], which has shown significant promise in addressing various combinatorial optimization problems with sequence generation tasks, such as the planar Travelling Salesman Problem (TSP) [22] and the Rectilinear Minimum Spanning Tree (RMST) problem [4].

Before diving into the details of our Selector and Conductor networks, we illustrate this **Selector** $\rightleftharpoons$ **Conductor** cooperation mechanism in Fig. 3 (right). To assist the Selector and Conductor in making decisions, we define hidden states $H_t^1 \in \mathbb{R}^{d_1}$ and $H_t^2 \in \mathbb{R}^{d_1}$ (initially set to $\vec{0}$) to encode the environment at step t , where d_1 is a hyperparameter. At step t , the Selector network selects a new hub node based on the hidden state H_t^1 . The action of selecting the hub node is encoded and can be passed to the Conductor to update its hidden state, resulting in H_t^2 , thereby assisting the Conductor in making decisions. If the Conductor chooses to continue selecting hub nodes, the Selector progresses to step $t + 1$. It is important to note that the encoding of the action at step t will be employed to update the hidden state of the Selector, yielding H_{t+1}^1 . Subsequently, the Selector will use H_{t+1}^1 to select hub nodes as described above.

Now, we introduce more details for the Selector and leave the details for the Conductor to Section III-C. At step t , leveraging H_t^1 , the probabilities of those n nodes being selected as the hub node can be obtained by the following n -dimensional vector:

$$p_t^{\text{sel}} = \text{Softmax}(\tanh(\text{Mask}(z_t^1))), \quad (6)$$

where the value $z_t^1 = g^{1\top} \tanh(W_1^1 E^{1\top} + W_2^1 H_t^1)$ with $W_1^1 \in \mathbb{R}^{d_2 \times d}$, $W_2^1 \in \mathbb{R}^{d_2 \times d_1}$, and $g^1 \in \mathbb{R}^{d_2}$ being the parameters to be trained (the integer d_2 is a pre-specified hyperparameter), and E^1 represents the node embeddings for the Selector network. The computation mechanism of Eq. 6 is based on the Pointer Network [22], and let us explain it in detail. The value $z_t^1 \in \mathbb{R}^n$ represents the logits for selecting different nodes. For the nodes that have already been selected, we use $\text{Mask}(z_t^1)$ to set their values to $-\infty$, thereby preventing them from being re-selected. The “softmax” function normalizes this vector and outputs the probabilities of those n nodes to be selected, i.e., the vector $p_t^{\text{sel}} \in \mathbb{R}^n$. Note that during the training phase, selections are made according to the probability distribution defined by p_t^{sel} , whereas in the testing phase, the model directly chooses the node with the highest probability.

Suppose the Selector network selects the i -th node ($1 \leq i \leq n - 1$). It is important to embed this action to assist subsequent decisions. To achieve this, we use $A_t^1 \in \mathbb{R}^{d_1}$ to encode the selection action. Since the selected hub node will

be connected to the root node, the encoding of the i -th node $E^1[i] \in \mathbb{R}^d$ should be combined with the encoding of the root node $E^1[0]$:

$$A_t^1 = W_a^1 E^1[i] + W_r^1 E^1[0], \quad (7)$$

where $W_a^1 \in \mathbb{R}^{d_1 \times d}$ and $W_r^1 \in \mathbb{R}^{d_1 \times d}$ are the parameters to be trained. $W_r^1 E^1[0]$ and $W_a^1 E^1[i]$ represent the transformed embeddings of the root node and the currently selected hub node, respectively. After the action is encoded, the hidden state of the Selector for step $t+1$ is updated as follows:

$$H_{t+1}^1 = \max(0, \max(H_t^1, W_u^1 A_t^1)), \quad (8)$$

where $W_u^1 \in \mathbb{R}^{d_1 \times d_1}$ is the parameter to be trained. The new hidden state H_{t+1}^1 is obtained by integrating the previous state H_t^1 with the current action encoding $W_u^1 A_t^1$.

C. Conductor Network

The Conductor network monitors the environment and assesses existing actions to determine whether to select additional hub nodes. Similar to the Selector, it also maintains a hidden state H_t^2 that records information from previous actions. At step t , the Conductor needs to perceive the action made by the Selector; thus, the selection action of the Selector at this step should be encoded to facilitate better decision-making by the Conductor. We employ the same method as illustrated in Eq. 7 and Eq. 8 to encode the action and update the hidden state of the Conductor, but with different trained parameters.

At step t , based on the hidden state H_t^2 , we compute:

$$z_t^2 = g^2 \tan\left(W_1^2 E^2 + W_2^2 H_t^2\right), \quad (9)$$

where $W_1^2 \in \mathbb{R}^{d_2 \times d}$, $W_2^2 \in \mathbb{R}^{d_2 \times d_1}$, and $g^2 \in \mathbb{R}^{d_2}$ are parameters to be trained, and E^2 represents the node embeddings for the Conductor network. The term $z_t^2 \in \mathbb{R}^n$ is expected to produce a vector that integrates information from the hidden state H_t^2 and the node embeddings. Based on z_t^2 , the probability of whether to continue selecting hub nodes at step t can be computed as:

$$p_t^{\text{con}} = \text{Sigmoid}\left(\vec{1} \cdot (\tanh(z_t^2))\right), \quad (10)$$

where $\vec{1} \cdot (\tanh(z_t^2))$ represents the sum of the elements of $\tanh(z_t^2)$. Eq. 10 aggregates information from different nodes to inform the decision on whether to continue selecting a hub node. If $p_t^{\text{con}} > 0.5$, the process proceeds to the next step; otherwise, the Selector will stop.

D. Critic and Training Method

In reinforcement learning, particularly in an Actor-Critic model, the Critic module is often used to assist the learning process [20]. Different from the previous RL-based networks [4], [5], our Critic module tries to predict the “cost-skew tradeoff value” of the input instance V , where the predicted value is denoted as $\text{Cr}(V)$. Recall that our Critic network receives the input $E^3 \in \mathbb{R}^{n \times d}$ from the Encoder, as described in Section III-A. Initially, we compute

an “attention-weighted” representation of E^3 : $\text{Attn}(E^3) = \text{softmax}(\tanh(E^3) g^3)^\top E^3$, where $g^3 \in \mathbb{R}^d$ is a trainable parameter vector, and $\text{Attn}(E^3)$ represents a weighted sum of the encoded information from different nodes. Then, two fully connected layers are utilized to obtain the predicted cost-skew value:

$$\text{Cr}(V) = \max\left(0, \text{Attn}(E^3) W_1^3 + \vec{b}_1^3\right) W_2^3 + b_2^3, \quad (11)$$

where $W_1^3 \in \mathbb{R}^{d \times d_3}$, $\vec{b}_1^3 \in \mathbb{R}^{d_3}$, $W_2^3 \in \mathbb{R}^{d_3 \times 1}$, and $b_2^3 \in \mathbb{R}$ are trainable parameters (with d_3 being a hyperparameter). After obtaining $\text{Cr}(V)$, we introduce the training method of the Actor and Critic networks.

Let θ_1 and θ_2 denote the parameters for the Actor and Critic, respectively. Suppose the batch size is $N \geq 1$ and the set of sampled instances for training are $\mathcal{V} = \{V_1, V_2, \dots, V_N\}$. For each $1 \leq i \leq N$, assume $\mathcal{T}_i$ is the trajectory executed by the Actor, which follows a Markov decision process. Each step consists of selecting a hub node and deciding whether to continue. The probability of trajectory $\mathcal{T}_i$ under the Actor’s policy is denoted as $p_{\theta_1}(\mathcal{T}_i)$. This probability is calculated as the product of the probabilities of the actions taken, including selecting hub nodes and making continuation decisions. The cost-skew value of the solution produced by the Actor is denoted by $\text{Ac}(V_i, \mathcal{T}_i)$ for each i . The loss for the Actor, denoted as $\mathcal{L}_A(\theta_1 | \mathcal{V})$, is calculated by:

$$\frac{1}{N} \sum_{i=1}^N \left[\text{Cr}(V_i) - \text{Ac}(V_i, \mathcal{T}_i) + \sum_{t=1}^{|\mathcal{T}_i|} R_t \gamma^t \right] \log p_{\theta_1}(\mathcal{T}_i | V_i), \quad (12)$$

where $|\mathcal{T}_i|$ denotes the number of hub nodes chosen by the Actor in trajectory $\mathcal{T}_i$. R_t and γ respectively represent the intermediate reward and the decay factor, which are used to balance the exploration and exploitation in RL [20].

The loss for the Critic $\mathcal{L}_C(\theta_2 | \mathcal{V})$ is based on the Mean Squared Error (MSE) function:

$$\frac{1}{N} \sum_{i=1}^N \|\text{Cr}(V_i) - \text{Ac}(V_i, \mathcal{T}_i)\|_2^2. \quad (13)$$

Both the Actor and Critic networks are trained using the Stochastic Gradient Descent (SGD) method [23].

IV. EXPERIMENTAL RESULTS

We trained our model using PyTorch and conducted our experiments on a 64-bit Linux system equipped with eight NVIDIA RTX A6000 GPUs. We compare the performance of our algorithm with three representative baselines: (1) the “BST-DME” algorithm outlined in [11]; (2) a heuristic multi-source selection method introduced in [19], denoted as “MSS-MSPD”; (3) the “PD” algorithm [18], [24] introduced in Section II-B. Our algorithm is labeled as “OURS”. We present our results separately for small-scale ($n \leq 50$) and large-scale ($n > 50$) instances.

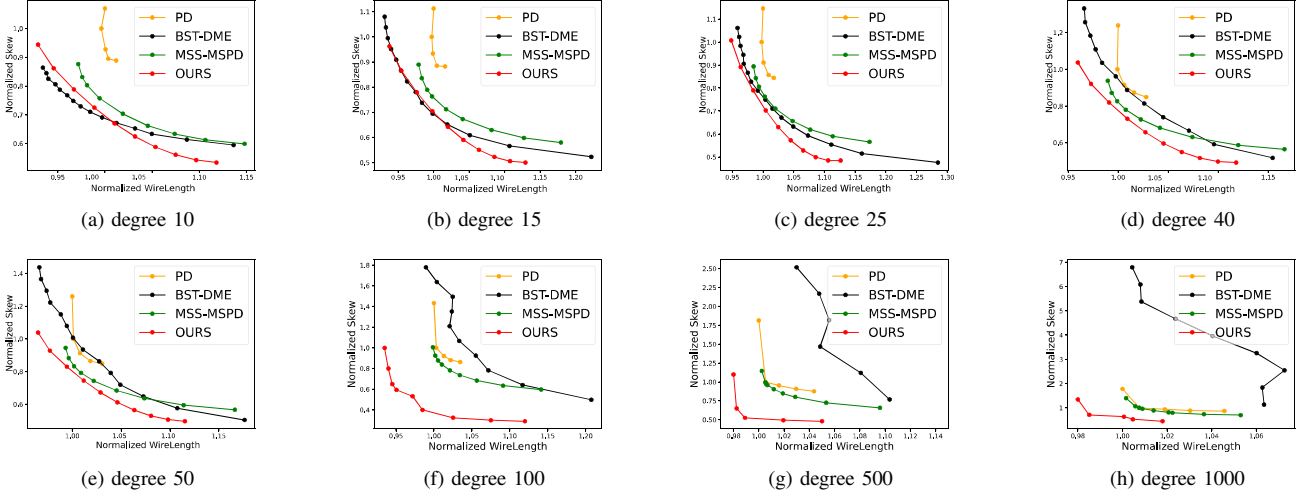


Fig. 4: Figures (a) to (e) show the comparison results of our framework with the BST-DME, MSS-MSPD, and PD algorithms for data with degrees of 10, 15, 25, 40, and 50, respectively. Figures (f) to (h) show the comparison results for large degrees of 100, 500, and 1000, respectively.

A. Small-Scale

We trained our models for degrees ranging from 10 to 50 on randomly generated training data. For degree 10, we trained the model with mini-batch size $B = 2048$ for 100k iterations. Subsequently, the batch size was diminished for each ascending degree. In this part, we compare the experimental performance for nets with degrees ≤ 50 , using the test data (300 instances for each degree) derived from the recent OpenROAD contest [25], which is generated by a well-designed random algorithm, ensuring the diversity of distribution.

The Prim’s algorithm and Dijkstra’s algorithm are commonly used representative algorithms for optimizing cost and skew [18], so we particularly take their obtained values “ $\text{cost}(T_P)$ ” and “ $\text{skew}(T_D)$ ” as the baseline values, where T_P and T_D are the trees obtained by Prim’s and Dijkstra’s algorithms, respectively. Suppose T is the tree obtained by an algorithm (either our method or a baseline), then we evaluate this algorithm’s performance by the ratios $\frac{\text{cost}(T)}{\text{cost}(T_P)}$ and $\frac{\text{skew}(T)}{\text{skew}(T_D)}$, where are respectively referred as “**normalized wirelength**” and “**normalized skew**”. The comparative performance of these algorithms is illustrated in Fig. 4 (a)-(e). It can be observed that our algorithm achieves the best Pareto frontiers, and the performance difference between “OURS” and the other algorithms gradually increases as the scale increases. To more clearly illustrate this trend, we also conducted the experiment on large scale instances in the next section.

B. Large-Scale

We compare the algorithms on randomly generated large-degree networks (degrees of 100, 500, and 1000, with 100 instances for each degree). The results are shown in Fig. 4 (f)-(h). It can be seen that the BST-DME algorithm has relatively poor performance on large-scale instances. Since the BST-DME algorithm has to obey skew constraints, the wirelength cost caused by this constraint accumulates rapidly with the

expansion of scale, and our framework maintains a superior performance relative to other algorithms. It is worth noting that we directly use the model trained in degree 100 for degrees 500 and 1000 without fine-tuning, which implies the effectiveness of applying our model to large-scale problems. The runtimes of various algorithms across different degrees are shown in Table I. Since most of the computational load of our learning-based model belongs to the training stage, the running time of solving a new instance in the test stage is quite small. Our runtime is comparable with PD and slightly slower than BST-DME, but our algorithm enjoys significantly better performance than these two baselines as shown in Fig. 4.

TABLE I: Average runtime performance over 1000 samples for small-scale tests and over 100 samples for large-scale tests.

Degree	BST-DME(s)	MSS-MSPD(s)	PD(s)	OURS(s)
10	0.022	0.007	0.001	0.003
15	0.045	0.074	0.005	0.007
25	0.057	0.107	0.009	0.009
40	0.081	0.240	0.014	0.015
50	0.094	0.400	0.018	0.019
100	0.202	2.575	0.103	0.104
500	1.216	136.6	1.387	1.415
1000	2.480	1008	4.392	5.362

V. CONCLUSION

In this paper, we propose an adaptive learning-based approach to select hub nodes for the cost-skew tradeoff problem. The key idea relies on a novel Selector $\rightleftharpoons$ Conductor cooperation mechanism integrated with the RL framework. Our experimental results illustrate the promising performance of our model on small and large-scale nets. In the future, we can consider further speeding up our method through other aspects. For example, it is interesting to consider whether the MSPD algorithm can be implemented on GPUs. It is also worth investigating whether our adaptive “Selector $\rightleftharpoons$ Conductor” RL framework can be applied to solve other challenging optimization problems in EDA.

REFERENCES

- [1] M. Hanan, "On steiner's problem with rectilinear distance," *SIAM Journal on Applied mathematics*, vol. 14, no. 2, pp. 255–265, 1966.
- [2] C. Chu and Y.-C. Wong, "Flute: Fast lookup table based rectilinear steiner minimal tree algorithm for vlsi design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 1, pp. 70–83, 2007.
- [3] M. D. Moffitt, "Maizerouter: Engineering an effective global router," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 11, pp. 2017–2026, 2008.
- [4] J. Liu, G. Chen, and E. F. Young, "Rest: Constructing rectilinear steiner minimum tree via reinforcement learning," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1135–1140.
- [5] L. Yang, G. Sun, and H. Ding, "Towards timing-driven routing: An efficient learning based geometric approach," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [6] R. Cheng, X. Lyu, Y. Li, J. Ye, J. Hao, and J. Yan, "The policy-gradient placement and generative routing neural networks for chip design," *Advances in Neural Information Processing Systems*, vol. 35, pp. 26 350–26 362, 2022.
- [7] V. Soteriou, N. Easley, H. Wang, B. Li, and L.-S. Peh, "Polaris: A system-level roadmap for on-chip interconnection networks," in *2006 International Conference on Computer Design*. IEEE, 2006, pp. 134–141.
- [8] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: from graph partitioning to timing closure*. Springer, 2011, vol. 312.
- [9] M. R. Guthaus, G. Wilke, and R. Reis, "Revisiting automated physical synthesis of high-performance clock networks," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 18, no. 2, pp. 1–27, 2013.
- [10] K. D. Boese and A. B. Kahng, "Zero-skew clock routing trees with minimum wirelength," in *[1992] Proceedings. Fifth Annual IEEE International ASIC Conference and Exhibit*. IEEE, 1992, pp. 17–21.
- [11] J. Cong, A. B. Kahng, C.-K. Koh, and C.-W. A. Tsao, "Bounded-skew clock and steiner routing," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 3, no. 3, pp. 341–388, 1998.
- [12] D. J. Huang, A. B. Kahng, and C.-W. A. Tsao, "On the bounded-skew clock and steiner routing problems," in *Proceedings of the 32nd annual ACM/IEEE Design Automation Conference*, 1995, pp. 508–513.
- [13] A. B. Kahng and C.-W. A. Tsao, "Practical bounded-skew clock routing," *High Performance Clock Distribution Networks*, pp. 87–103, 1997.
- [14] M. R. Garey and D. S. Johnson, "The rectilinear steiner tree problem is np-complete," *SIAM Journal on Applied Mathematics*, vol. 32, no. 4, pp. 826–834, 1977.
- [15] A. Z. Zelikovsky and I. I. Mandoiu, "Practical approximation algorithms for zero-and bounded-skew trees," *SIAM Journal on Discrete Mathematics*, vol. 15, no. 1, pp. 97–111, 2001.
- [16] M. Charikar, J. Kleinberg, R. Kumar, S. Rajagopalan, A. Sahai, and A. Tomkins, "Minimizing wirelength in zero and bounded skew clock trees," *SIAM Journal on Discrete Mathematics*, vol. 17, no. 4, pp. 582–595, 2004.
- [17] K. Han, A. B. Kahng, C. Moyes, and A. Zelikovsky, "A study of optimal cost-skew tradeoff and remaining suboptimality in interconnect tree constructions," in *Proceedings of the 20th System Level Interconnect Prediction Workshop*, 2018, pp. 1–8.
- [18] C. J. Alpert, T. C. Hu, J.-H. Huang, A. B. Kahng, and D. Karger, "Prim-dijkstra tradeoffs for improved performance-driven routing tree design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 14, no. 7, pp. 890–896, 1995.
- [19] A. B. Kahng, S. Thumathy, and M. Woo, "An effective cost-skew tradeoff heuristic for vlsi global routing," in *2023 24th International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2023, pp. 1–8.
- [20] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [22] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," *Advances in neural information processing systems*, vol. 28, 2015.
- [23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [24] C. J. Alpert, W.-K. Chow, K. Han, A. B. Kahng, Z. Li, D. Liu, and S. Venkatesh, "Prim-dijkstra revisited: Achieving superior timing-driven routing trees," in *Proceedings of the 2018 International Symposium on Physical Design*, 2018, pp. 10–17.
- [25] TILOS-AI-Institute, "Mspd," Github, 2023. [Online]. Available: <https://github.com/TILOS-AI-Institute/multi-source-Prim-Dijkstra>